EECS 348 Assignment 4 - GenAI Analysis

Author: Charvi Reddy Konudula **Date:** October 15, 2026

a. Names of the two GenAI programs

- **ChatGPT** (OpenAI)
- **Gemini** (Google)

b. How I accessed them

- **ChatGPT:** through a web browser at chatgpt.com.
- **Gemini:** through a web browser at gemini.google.com.

c. Prompt used (identical for both)

> Write a single-file C++ program (Assignment4.cpp) that solves Sudoku puzzles using an object-oriented design (a class with member functions, not free functions) and a recursive depth-first backtracking search. The program reads five files, puzzle1.txt through puzzle5.txt, each a 9x9 grid of space-separated digits 1-9 where `_` marks a blank. For each file, print the file name, the puzzle as read, and then every solution found, or "No solution found" if none exists. If a puzzle has multiple solutions, print all of them. Compile with g++ on Linux.

d. Generated code (raw, unedited)

d.1 ChatGPT

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
using namespace std;

class Sudoku {
private:
    int grid[9][9];
    int solutionCount;

public:
    Sudoku() {
        solutionCount = 0;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                grid[row][col] = 0;
            }
        }
    }

    bool readPuzzle(const string& filename) {
        ifstream file(filename);

        if (!file.is_open()) {
            cerr << "Error opening file: " << filename << endl;
            return false;
        }

        string value;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (!(file >> value)) {
                    cerr << "Invalid puzzle format in "
                         << filename << endl;
                    return false;
                }

                if (value == "_") {
                    grid[row][col] = 0;
                } else {
                    try {
                        size_t pos;
                        int digit = stoi(value, &pos);

                        if (pos != value.length() ||
                            digit < 1 || digit > 9) {
                            cerr << "Invalid value in "
                                 << filename << endl;
                            return false;
                        }

                        grid[row][col] = digit;
                    } catch (...) {
                        cerr << "Invalid value in "
                             << filename << endl;
                        return false;
                    }
                }
            }
        }

        return true;
    }

    void printPuzzle() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (grid[row][col] == 0) {
                    cout << "_";
                } else {
                    cout << grid[row][col];
                }

                if (col < 8) {
                    cout << " ";
                }
            }
            cout << endl;
        }
```

```
    }

    bool isValid(int row, int col, int num) const {
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == num ||
                grid[i][col] == num) {
                return false;
            }
        }

        int boxRow = (row / 3) * 3;
        int boxCol = (col / 3) * 3;

        for (int r = boxRow; r < boxRow + 3; r++) {
            for (int c = boxCol; c < boxCol + 3; c++) {
                if (grid[r][c] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {
                if (grid[row][col] == 0) {
                    return true;
                }
            }
        }

        return false;
    }

    void solve() {
        int row, col;

        if (!findEmptyCell(row, col)) {
            solutionCount++;
            cout << "\nSolution " << solutionCount << ":" << endl;
            printPuzzle();
            return;
        }

        for (int num = 1; num <= 9; num++) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;

                solve();

                // Backtrack to search for other solutions.
                grid[row][col] = 0;
            }
        }
    }

    void solvePuzzle() {
        solutionCount = 0;
        solve();

        if (solutionCount == 0) {
            cout << "No solution found" << endl;
        } else {
            cout << "\nTotal solutions found: "
                 << solutionCount << endl;
        }
    }
};

int main() {
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";

        cout << "\n==================================" << endl;
        cout << "File: " << filename << endl;
        cout << "==================================" << endl;

        Sudoku puzzle;

        if (puzzle.readPuzzle(filename)) {
            cout << "\nOriginal puzzle:" << endl;
            puzzle.printPuzzle();

            cout << "\nSolving puzzle..." << endl;
            puzzle.solvePuzzle();
        }
        cout << endl;
```

```
    }
    return 0;
}
```

d.2 Gemini

```
// Assignment4.cpp
#include <iostream>
#include <fstream>
#include <string>

class SudokuSolver {
private:
    char board[9][9];
    char original[9][9];

    bool isValid(int row, int col, char num) const {
        for (int i = 0; i < 9; ++i) {
            if (board[row][i] == num) return false;
            if (board[i][col] == num) return false;
            if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == num) return false;
        }
        return true;
    }

    void solveRecursive(int row, int col, int& solutionCount) {
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            printCurrentBoard();
            return;
        }

        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        if (board[row][col] != '_') {
            solveRecursive(nextRow, nextCol, solutionCount);
            return;
        }

        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                board[row][col] = num;
                solveRecursive(nextRow, nextCol, solutionCount);
                board[row][col] = '_'; // Backtrack
            }
        }
    }

    void printCurrentBoard() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << board[i][j] << (j == 8 ? "" : " ");
            }
            std::cout << "\n";
        }
        std::cout << "\n";
    }

public:
    bool loadPuzzle(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) return false;
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                file >> board[i][j];
                original[i][j] = board[i][j];
            }
        }
        return true;
    }

    void printOriginal() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << original[i][j] << (j == 8 ? "" : " ");
            }
            std::cout << "\n";
        }
    }

    void solve() {
        int solutionCount = 0;
        solveRecursive(0, 0, solutionCount);
        if (solutionCount == 0) {
            std::cout << "No solution found.\n\n";
        }
    }
};

int main() {
```

```
    for (int i = 1; i <= 5; ++i) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        std::cout << "File: " << filename << "\n";
        SudokuSolver solver;
        if (solver.loadPuzzle(filename)) {
            std::cout << "Puzzle as read:\n";
            solver.printOriginal();
            std::cout << "\nSolutions:\n";
            solver.solve();
        } else {
            std::cout << "Could not open file " << filename << "\n\n";
        }
        std::cout << "----------------------------------------\n";
    }
    return 0;
}
```

e. Analysis of the two programs

How I tested. I compiled both programs with `g++ -std=c++17 -O2` and ran them on puzzle1.txt through puzzle5.txt. Because a single run takes only a few milliseconds, I also timed 200 back-to-back runs of each program. I diffed the sets of solved grids and checked behavior on a deliberately truncated puzzle file.

Correctness

Puzzle	ChatGPT	Gemini
puzzle1.txt	1 solution (matches solution1.txt)	1 solution (matches solution1.txt)
puzzle2.txt	1 solution	1 solution
puzzle3.txt	1 solution	1 solution
puzzle4.txt	No solution found	No solution found.
puzzle5.txt	2 solutions	2 solutions

- Both programs print identical sets of solved grids for all five puzzles, and both are recursive and object-oriented (a class with member functions).
- **Input handling differs.** ChatGPT checks that the file opens, that 81 symbols are present, and that each symbol is '_' or a digit 1-9, and it prints an error otherwise. Gemini does not check anything after opening the file. When I fed it a file cut short after about 60 characters, it still printed 'Puzzle as read' using leftover or uninitialized values instead of reporting an error. Gemini also does not reject bad symbols such as 0 or letters.
- Output: ChatGPT prints a total solution count; Gemini does not. Gemini's 'No solution found.' has a trailing period, so it does not match the assignment wording exactly.

Execution time

Measurement (g++ -O2)	ChatGPT	Gemini
200 runs of all five puzzles	0.345 s total (about 1.7 ms per run)	0.394 s total (about 2.0 ms per run)

- The two are essentially tied in practice, and the difference is within noise because process start-up dominates a run this short.
- Algorithmically, both are depth-first backtracking that tries digits 1-9 in order, so both are exponential in the worst case. ChatGPT's findEmptyCell() rescans the grid from cell (0,0) on every recursive call, adding up to 81 extra cell checks per call. Gemini walks the cells in order and never rescans, which is cheaper per call.

Space complexity

- ChatGPT stores one 9x9 grid of int, which is 324 bytes. Gemini stores two 9x9 char arrays (board and original), which is 162 bytes.
- Both use recursion depth of at most 81 (one frame per empty cell), so extra stack space is O(1) for a fixed 9x9 board. Neither uses dynamic memory.
- ChatGPT includes but never uses it.

Maintainability

- **ChatGPT:** well structured, with separate methods for reading, printing, validation, finding empty cells, and solving, and a solvePuzzle() wrapper. Weaknesses: using namespace std, magic numbers (9, 3), an unused include, and solve() prints directly while also being the search routine.
- **Gemini:** shorter and uses std:: explicitly. The isValid() check is compact but its block-index arithmetic is hard to read, there are no comments except 'Backtrack', and the lack of error handling makes failures hard to diagnose. Magic numbers also appear throughout.

f. Program selected and justification

I selected ChatGPT's program as the basis for my assignment. The evidence from section e:

- Correctness is the deciding factor. Both produce identical, correct answers on all five puzzles, but only ChatGPT detects a short or malformed file and reports an error. Gemini silently continues with bad data.
- Execution time does not separate them: 1.7 ms versus 2.0 ms per run is noise, so speed alone is no reason to prefer Gemini.
- ChatGPT's code is already split into small, single-purpose methods and tracks a solution count, which gives a better starting structure to extend.
- Its weaknesses (full-grid rescans, int storage, magic numbers) are all easy and low-risk to fix, whereas adding validation to Gemini would mean adding a missing feature.

g. Improvements I made

Area	What I changed	Result
Correctness	Kept ChatGPT's input validation (file opens, 81 cells, symbols limited to '_' and 1-9). Changed the no-solution message to print exactly 'No solution found'. Kept the all-solutions search and total count. The program keeps an untouched copy of the puzzle so 'the puzzle from the file' always prints as read.	Same correct answers on all five puzzles (1, 1, 1, none, 2), verified against solution1.txt and on the cycle server. Bad or short files produce an error, which Gemini's version does not.
Execution time	Replaced findEmptyCell() with Gemini's cell-by-cell traversal (one cell index passed down the recursion), so the program never rescans from the top. Merged the row, column, and block checks into one loop of 9 iterations instead of separate loops.	Removes up to 81 cell checks per recursive call and about 9 loop passes from each validity test. Measured total time over 200 runs stayed about 0.35 s, so the gain is real in theory but too small to show on these puzzles.
Space complexity	Changed the grid from int to char (81 bytes for the working board). The saved original copy brings total board storage to 162 bytes, down from 324 bytes.	Board memory halved. Recursion depth is unchanged at 81 maximum, which is the minimum for this approach.
Maintainability	Replaced magic numbers with named constants (SIZE, BOX, EMPTY, NUM_PUZZLES). Removed using namespace std and the unused include. Separated the search (search()), the legality test (isSafe()), and the printing (printBoard()). Added a prologue and a comment on every line.	The code is easier to read and change, for example changing the board size means editing one constant.